

17장 강화학습

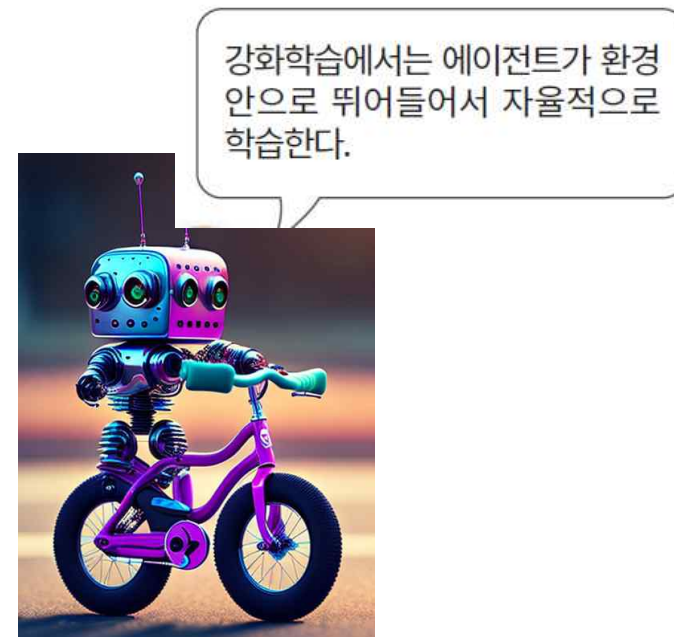
학습 목표

- 전통 Q-학습을 이해한다.
- 심층 Q-학습 신경망을 이해한다.
- 심층 Q-학습 신경망의 문제점, 타겟 신경망을 알아본다.

가치 기반과 정책 기반 강화학습이 있
가치 기반이 오래 된 것

강화학습이란?

- 지금까지 우리가 살펴본 딥러닝에서는 항상 훈련 데이터와 정답 레이블이 있었다.
- 만약 딥러닝을 탑재한 에이전트가, 환경에서 스스로 행동하여서 학습할 수 있다면 어떨까?



알파스타

- 스타크래프트는 불완전한 정보를 가지고 있고, 실시간으로 경기가 진행되며, 장기 계획이 필요한 어려운 게임이다.
- 하지만 스타크래프트에서도 강화 학습 인공지능이 인간을 5-0으로 물리친 바 있다.

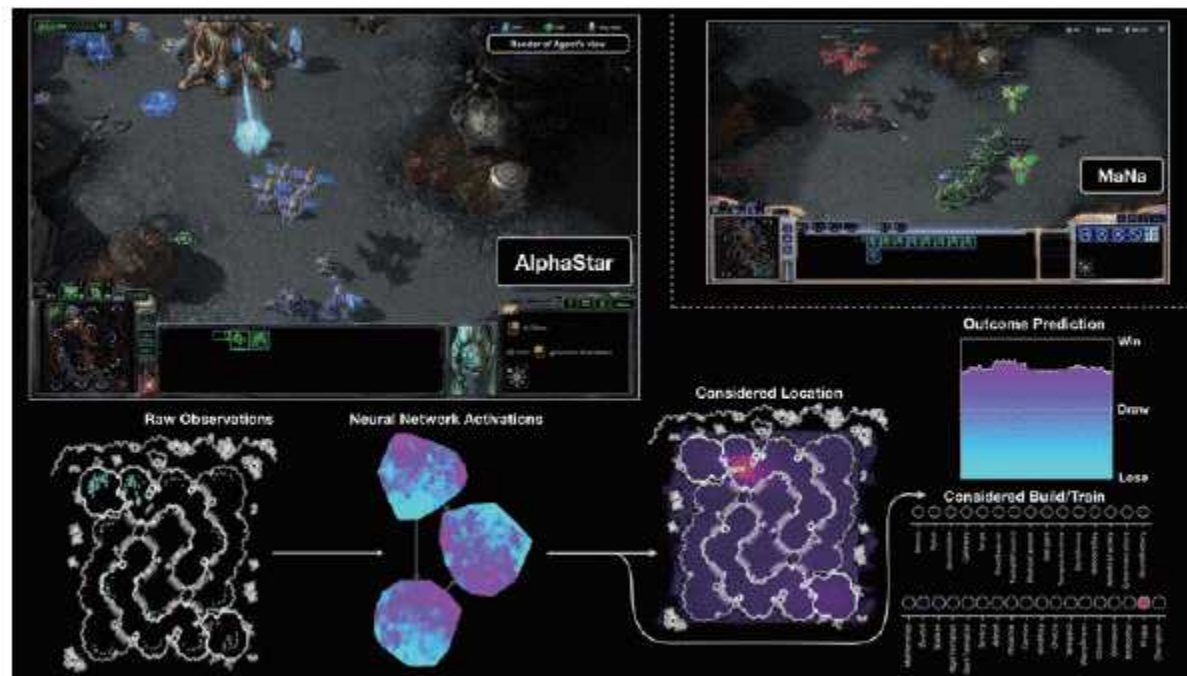
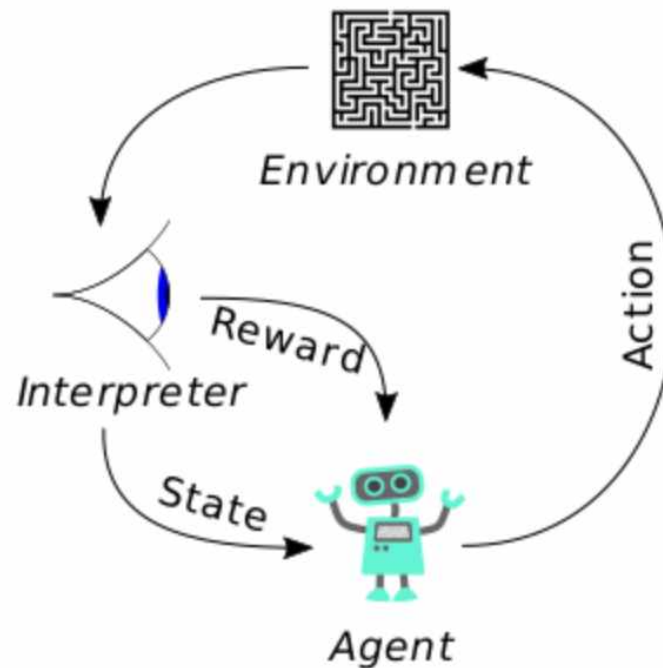


그림 13-2 스타크래프트 게임을 수행하는 알파스타

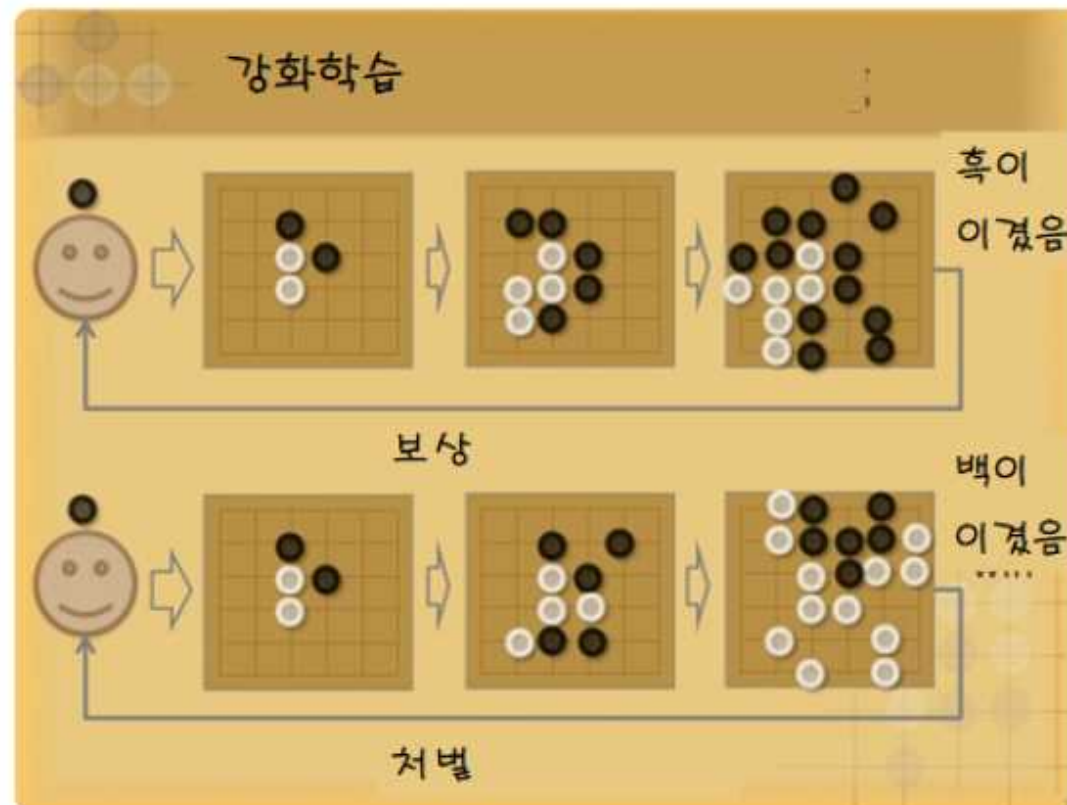
강화학습의 기본 원리

- 강화 학습(**Reinforcement Learning**)에서는 에이전트가 환경 안에서 어떤 행동을 취할 때마다 외부에서 처벌이나 보상이 주어진다. 에이전트는 이 보상을 최대화하는 방향으로 학습을 진행시킨다.



알파고에서의 강화 학습

- 최신 버전의 알파고에서는 예전의 기보를 학습하는 것이 아니라 바둑의 규칙만을 알려준 다음, 2대의 알파고끼리 바둑을 두게 한다. 좋은 바둑 수를 둔 알파고 한테는 보상이 주어진다. 이 보상을 이용하여 학습이 진행되는 것이다



강화 학습 프레임워크

- 에이전트(agent): 강화 학습의 중심이 되는 객체
- 환경(Environment): 에이전트가 작동하는 물리적 세계
- 상태(state): 에이전트의 현재 상황, 미로에서의 에이전트의 위치가 상태일 수 있다.
- 보상(reward) : 환경으로부터의 피드백,
- 액션(action) : 에이전트의 행동

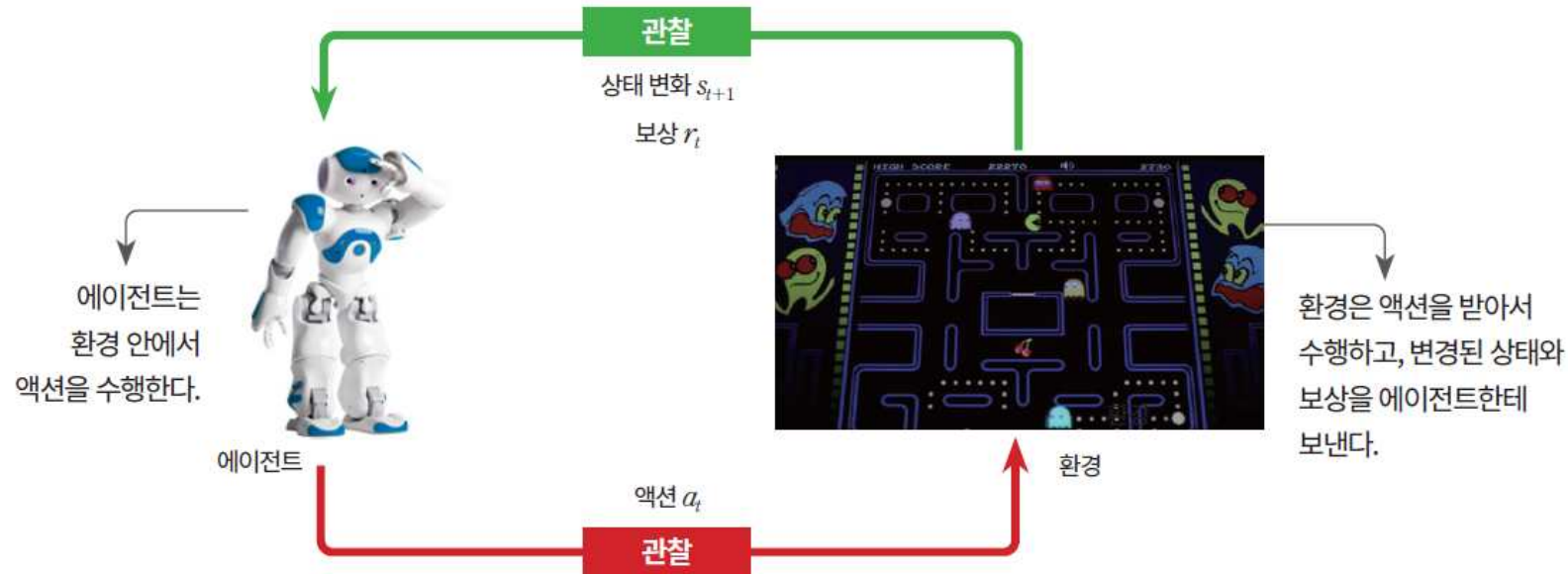


그림 17-5 강화 학습의 환경
바뀐 상태와 그에 따른 보상을 해준다, 그걸 통해 나의 행동을 바

게임에서의 강화학습

- 강화 학습에서 에이전트는 환경 안에서 자신의 보상을 극대화하려고 한다. 보상은 성공 또는 실패에 대한 피드백이다. 에이전트가 행동할 때마다 보상을 받을 필요는 없지만, **보상이 지연될 수 있다**. 즉 마지막에 하나의 보상만을 받는 경우도 많다.

문제가 있다, 보상이 바로 안오는 문제가 있다.

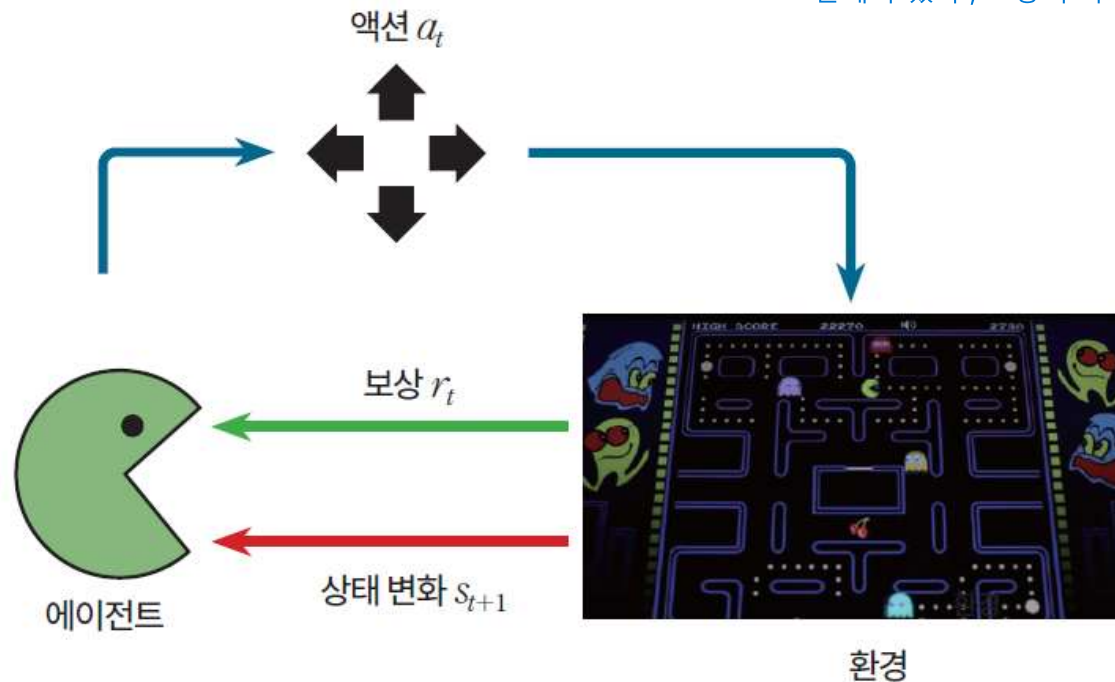


그림 17-6 팩맨에서의 강화 학습

강화학습과 딥러닝

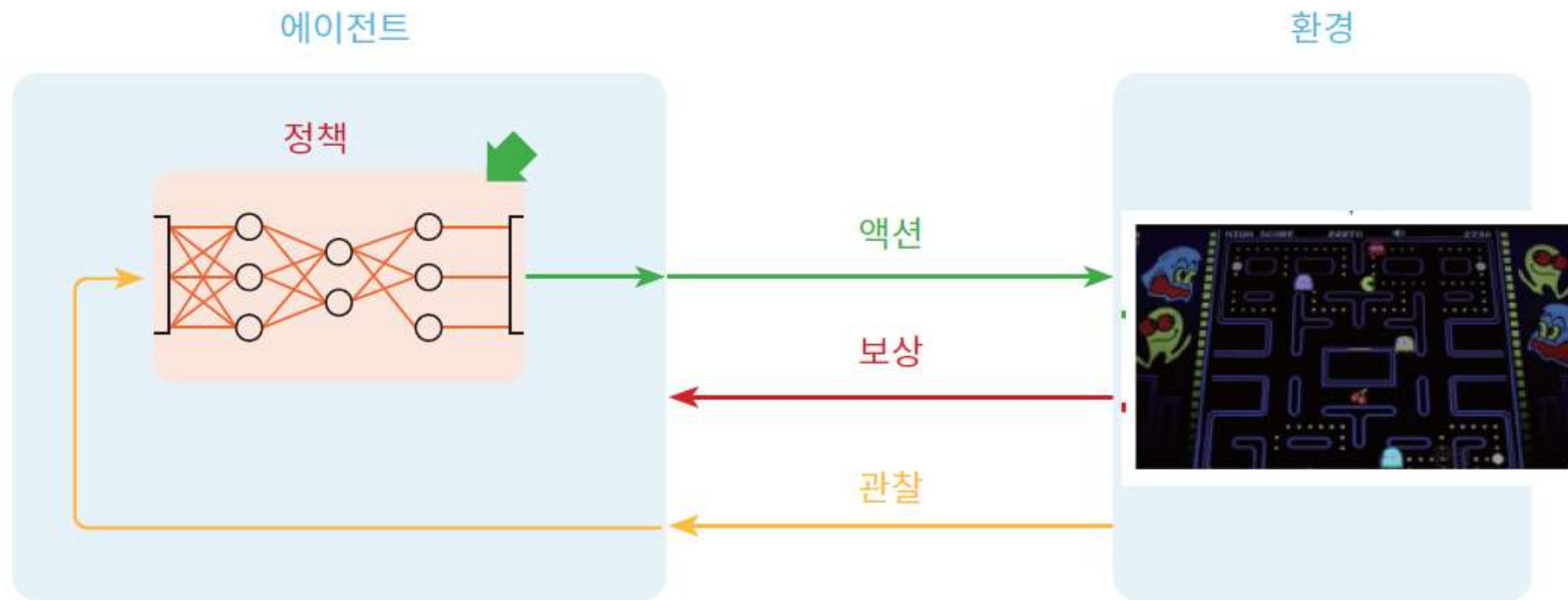


그림 17-7 팩맨에서의 강화 학습

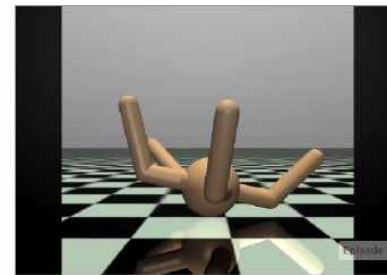
Reinforcement Learning

구분	핵심 아이디어	대표 알고리즘	언제 쓰나?
강화학습 전체	에이전트가 환경과 상호작용하며 보상 최대화 행동을 스스로 배운다		게임 AI, 로봇, 자율주행, 광고 추천, 주식 트레이딩, 의료 정책
Q-Learning 계열 (Value-based)	상태-행동 가치 $Q(s,a)$ 를 학습 $\rightarrow$ 행동은 argmax 로 선택	DQN, Double DQN, Rainbow	상태, 행동이 이산적이고 적을 때 (게임, 보드게임, 클릭 예측)
Policy Learning 계열 (Policy-based)	정책 $\pi_{\theta}(a s)$ 를 직접 학습 $\rightarrow$ 행동을 확률로 바로 출력	REINFORCE, PPO, SAC, A2C/A3C, TRPO	
Actor-Critic (하이브리드)	정책(Actor) + 가치함수(Critic)를 동시에 학습 $\rightarrow$ 안정성+성능 둘 다 잡음	A2C, A3C, PPO, SAC, TD3	현재 실무 80% 이상 차지

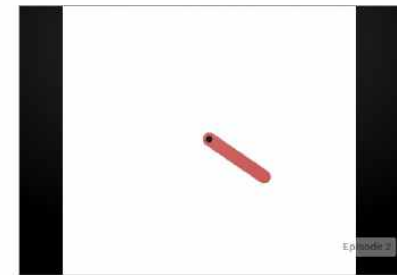
OpenAI

- OpenAI 재단은 인공지능을 위한 여러 가지 프로젝트를 진행하는 재단이다. OpenAI 재단은 초거대 AI인 GPT-4, 5.1을 만들어서 여러 가지 서비스를 하고 있다. 키워드를 주면 키워드로 이미지를 생성하는 DALL-E도 유명하다. OpenAI의 여러 서비스 중에서 강화학습을 위한 Gymnasium 라이브러리 (<http://www.openai.com/>)가 있다. Gymnasium은 강화학습 알고리즘을 개발하고 비교하기 위한 툴킷이다.

<https://github.com/openai/retro>
<https://openai.com/research/procgen-benchmark>



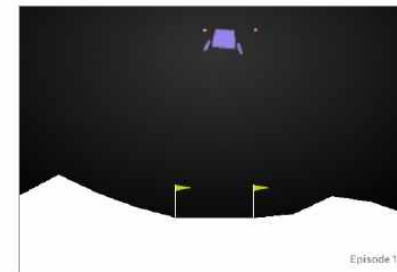
(a) 랜덤 보행 게임



(b) CartPole 게임



(c) 스페이스 인베이더 게임



(d) 루나 랜더 게임

그림 17-8 Gym 라이브러리가 제공하는 다양한 게임들

얼음 호수 게임

- 얼음 호수(frozen lake) 문제를 가지고 강화 학습을 설명하고자 한다. 얼음 호수 문제는 Open-AI 프레임워크에도 포함되어 있어서 파이썬으로 프로그래밍하기도 편리하다.



그림 17-9 얼음 호수 문제(frozen lake)

에이전트의 가능한 동작

- 에이전트에는 4가지의 가능한 액션(LEFT, DOWN, RIGHT, UP)이 있다.

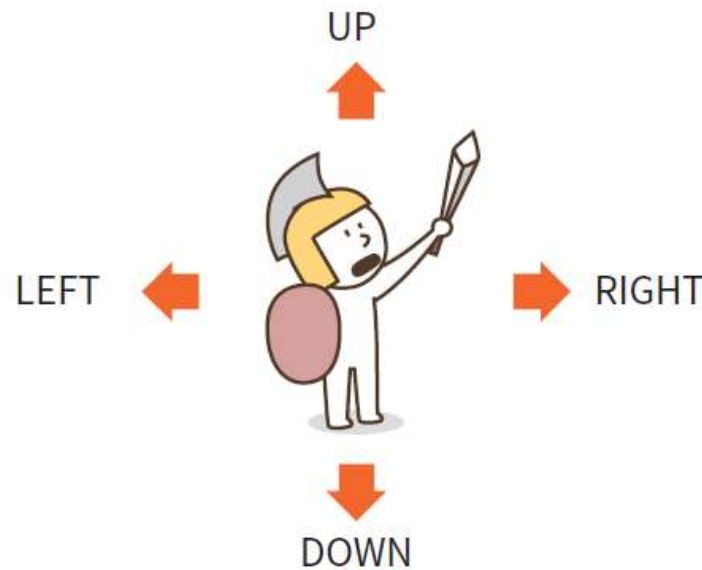


그림 17-10 에이전트의 행동

얼음 호수 게임 버전 #1

```
import gymnasium as gym                # (1)
import pygame
env = gym.make("FrozenLake-v1", render_mode='human', is_slippery=False) # (2)
print(env.observation_space.n)         # (3)
print(env.action_space.n)
env.reset()                            # (4)

n=30
env.render()
for i in range(n):                     # (5)
    action = env.action_space.sample() # (6)
    state, reward, done, truncated, info = env.step(action) # (7)
    print(f"({action}, {state}, {reward})", end=">")
    env.render()                       # (8)
    if done: break                     # (9)

env.close()
```

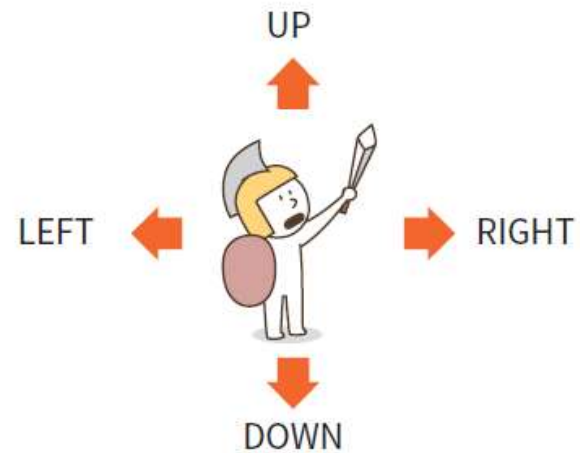
```
16
4
(2, 1, 0.0)->(0, 0, 0.0)->(2, 1, 0.0)->(1, 5, 0.0)->
```

얼음 호수 게임 버전 #1



상태와 액션

0(S)	1	2	3
4	5(H)	6	7(H)
8	9	10	11(H)
12(H)	13	14	15(G)



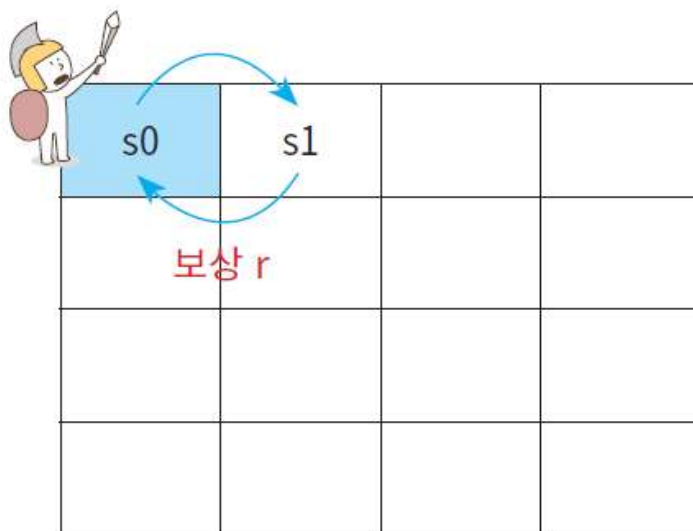
실행 결과 분석



그림 17-12 랜덤한 행동을 하는 얼음 호수 문제의 실행 결과

Q-학습 #1

- 이 게임은 아주 간단해 보이지만, 아무것도 모르는 에이전트 입장에서는 결코 만만한 문제가 아니다. 우리는 전체 게임 보드를 볼 수 있지만, 에이전트는 현재 있는 장소밖에는 알지 못한다. 에이전트는 어떤 행동을 해야만 다음 상태에 대하여 알 수 있다. 현재 에이전트가 시작 상태인 **s0**에 서 있다고 가정하자.



동적 프로그래밍(dynamic programming)

- 에이전트가 액션을 하고 보상을 받았으면, 이것을 기록으로 남겨서 다음번에 이용할 수 있도록 해야 하는 것이 좋을 것이다.
- 전통적인 방법은 ‘동적 프로그래밍(dynamic programming)’이라고 불리는 방법으로, 기본적으로 복잡한 문제를 ‘약간씩 겹치는 서브 문제’들로 분해하고 이들 서브 문제들의 결과를 테이블에 저장하는 방법이다. 에이전트가 어떤 상태에서 특정한 행동을 하고 보상을 받을 때마다 테이블에 기록한다.
 - 동적 프로그래밍은 이전 계산 결과를 저장해 반복 계산을 줄이는 알고리즘 기법이다
- 에이전트가 시행착오를 거듭할수록 테이블은 점점 정확해진다.

Q-테이블

상태(state)	LEFT ($\leftarrow$)	DOWN ($\downarrow$)	RIGHT ($\rightarrow$)	UP ($\uparrow$)
0	$Q(0, \leftarrow)$	$Q(0, \downarrow)$	$Q(0, \rightarrow)$	$Q(0, \uparrow)$
1	$Q(1, \leftarrow)$	$Q(1, \downarrow)$	$Q(1, \rightarrow)$	$Q(1, \uparrow)$
2	$Q(2, \leftarrow)$	$Q(2, \downarrow)$	$Q(2, \rightarrow)$	$Q(2, \uparrow)$
...				
14	$Q(14, \leftarrow)$	$Q(14, \downarrow)$	$Q(14, \rightarrow)$	$Q(14, \uparrow)$
15	$Q(15, \leftarrow)$	$Q(15, \downarrow)$	$Q(15, \rightarrow)$	$Q(15, \uparrow)$

보상

- Q 값은 근본적으로 보상에서 출발한다. 따라서 에이전트가 한 번이라도 우연히 목표 상태로 가야만이 보상이 시작되고 Q-테이블이 채워지기 시작한다. 그전까지는 모든 값은 0일 것이다.
- 얼음 호수 문제에서 보상은 어떻게 주어질까? 보상은 에이전트가 목표 타일에 도착해야만 주어진다. 이때만 보상이 1이 된다. 나머지 위치에서는 보상은 전부 0이 된다. 목표 타일로 가지 못하면 보상은 항상 0이다. 한 상태에서만 긍정적인 보상을 줄 수 있기 때문에, 에이전트는 상당히 판단하기 어렵다.

Q- 함수

- Q 함수는 상태 s 에 있는 에이전트가 어떤 행동 a 를 실행하여서 얻을 수 있는 총 보상값이다.
- 우리는 Q값을 반환하는 Q 함수를 생각할 수 있다. Q 함수는 에이전트의 현재 상태와 에이전트가 실행하는 행동을 받아서 총 보상값을 반환하는 함수이다.



그림 17-13 Q 함수는 현재의 상태와 특정 액션을 받아서 총 보상값을 반환한다.

정책

- 에이전트는 어떤 방향으로 이동하여야 할까? 4가지의 행동 LEFT, DOWN, RIGHT, UP에 대한 Q 값들을 모두 알 수 있다. 이 중에서 $Q(s, \text{RIGHT})$ 이 가장 높은 값이다. 이것이 의미하는 바는 현재 상태에서 오른쪽으로 이동하면 보상이 가장 높을 거라는 것이다. 이것을 강화학습에서는 정책(policy)이라고 한다. 가장 상식적인 정책은 Q 함수값 중에서 최대값을 찾고 최대값과 관련된 액션을 실행하는 것이다. 이것을 수학 기호를 써서 다음과 같이 표시한다.

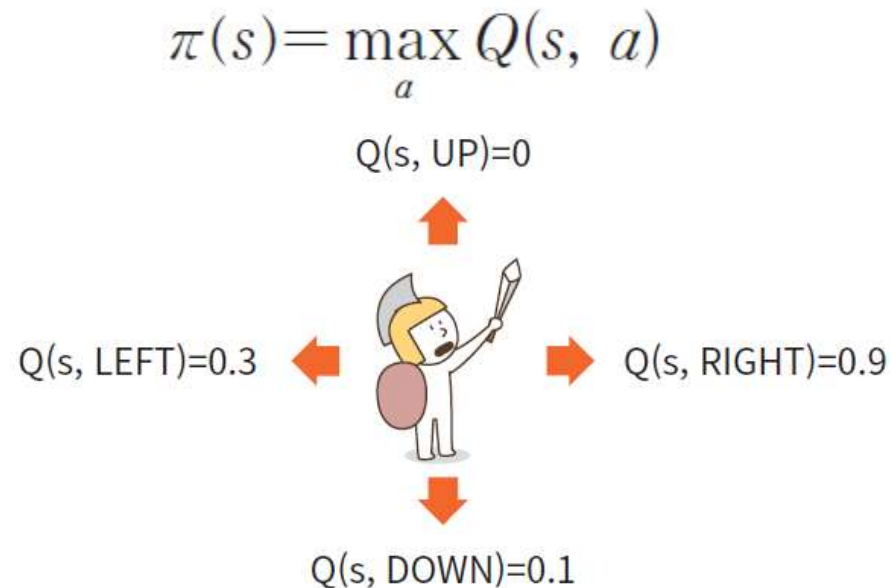


그림 17-14 어떤 상태에서 모든 액션에 대한 Q값을 알고 있을 때 액션을 결정하는 방법이 정책이다.

Q값 순환 관계식

- Q값은 어떤 상태에서 어떤 행동을 한 경우, 후속 행동으로 받을 수 있는 모든 보상을 더한 값이다. 따라서 옆 칸의 Q값과 어떤 관계가 있을까? 상태 **s**에서의 Q값은 다음과 같이 순환적으로 계산할 수 있다.

$$Q(s, a) = r + \max_{a'} Q(s', a')$$

Q 값 순환 관계식

- 총 보상은 다음과 같이 순환적으로 계산할 수 있다.

$$R_t = r_t + r_{t+1} + \dots + r_n$$

$$R_t = r_t + R_{t+1}$$

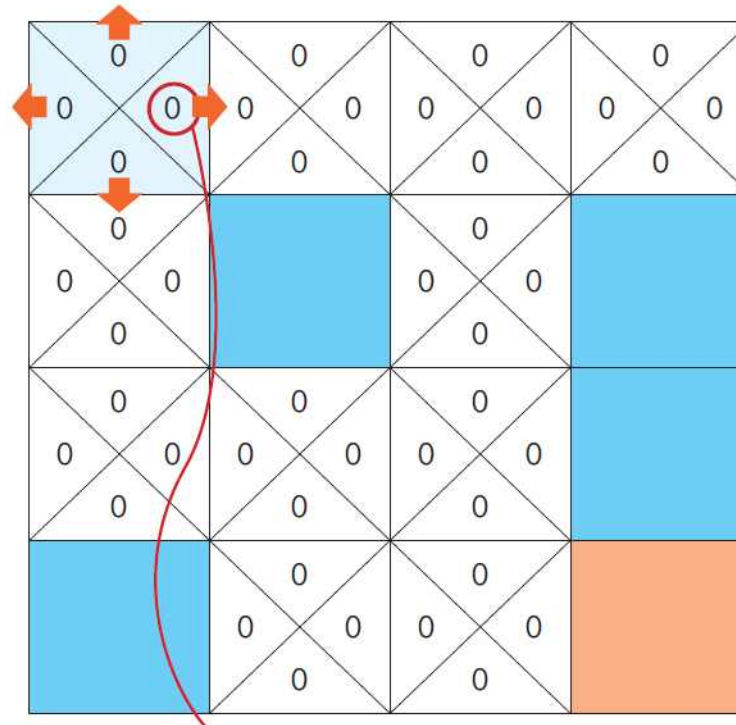
- 이것과 유사하게 상태 s 에서의 Q 값은 다음과 같이 순환적으로 계산할 수 있다. 즉 상태 s 에서 액션 a 를 실행하였을 때 받는 보상 r 에, 다음 상태에서의 Q 값 중에서 최대값을 더하게 된다.

$$Q(s, a) = r + \max_{a'} Q(s', a')$$

그 바뀐 상태에서 또 얻을 수 있는 보상 값을 더해줘야지 총 보상이

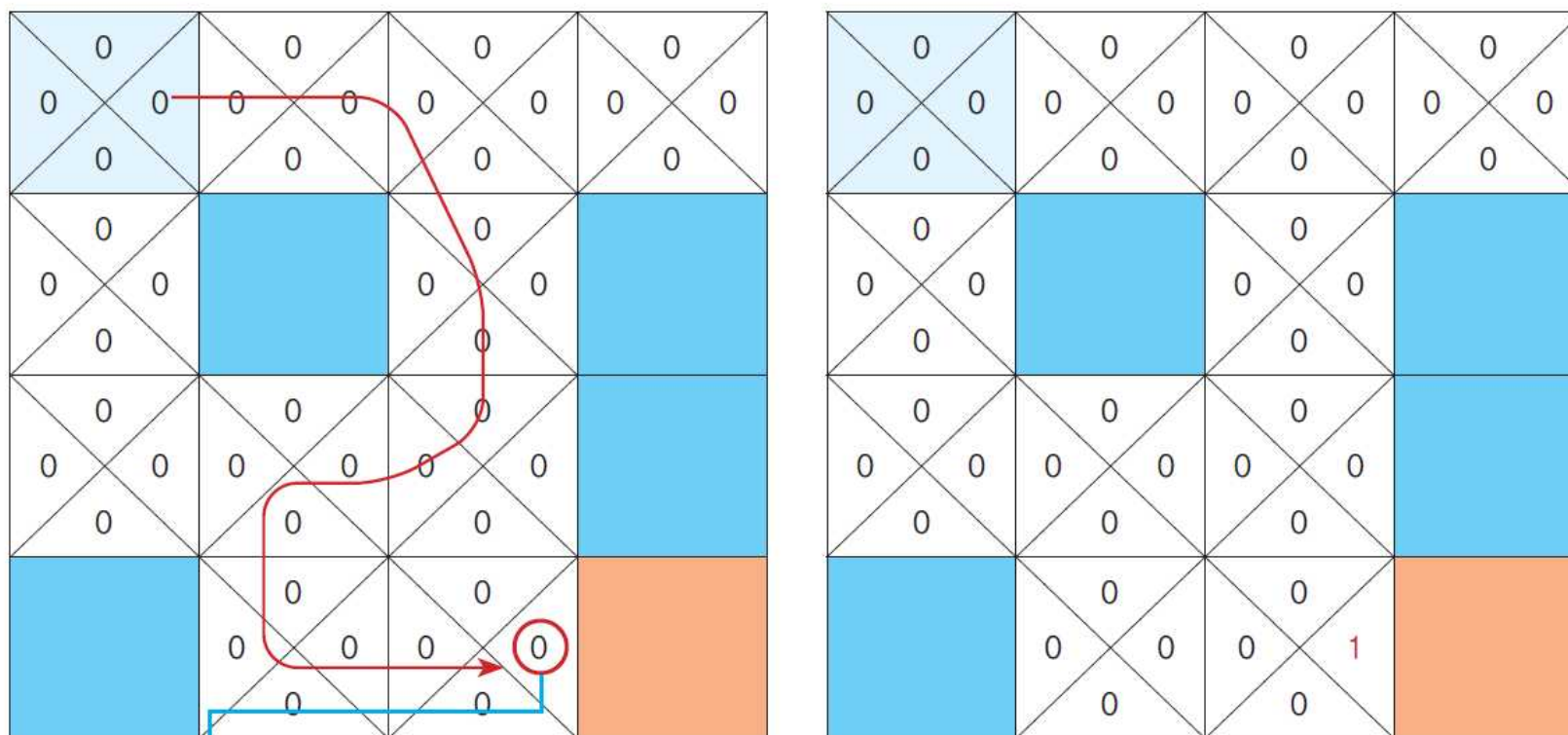
가장 중요한 수식이다. 전통적인 Q-학습에서는 결국 이 순환 관계식을 사용하여 테이블 내의 Q 값들이 계속 업데이트된다

얼음 호수 문제에서 실제로 Q-값을 계산해보자.



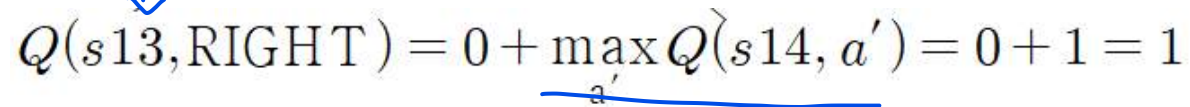
$$Q(s_0, \text{RIGHT}) = 0 + \max_{a'} Q(s_1, a') = 0 + 0 = 0$$

우연히 목표 상태로 가야만이 학습이 시작된다.



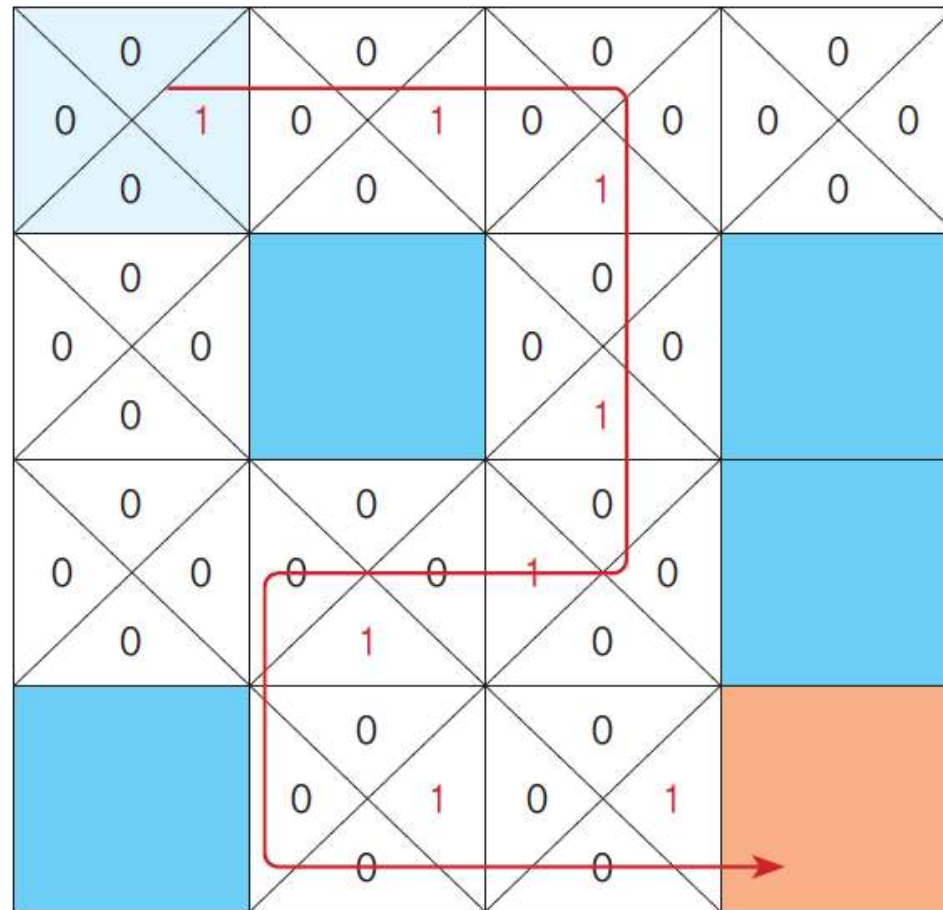
$Q(s_{14}, RIGHT) = 1 + \max_a Q(s_{15}, a') = 1 + 0 = 1$

오른쪽으로 이동한 보상 값이 1



28

Q-테이블 업데이트



점점 전달이 되면 해결책을 찾을 수 있다, 이것이 q테이블 학습방법
 이게 문제가 있다 최단경로를 못찾는다. 어떻게 하면 찾을 수 있을까?
 돌아가는 길을 보상 값을 조금 더 낮춘다 그것이 할인된 보상

얼음 호수 게임 버전 #2

```
import numpy as np
import gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=False) # (1)

states = env.observation_space.n           # = 16
actions = env.action_space.n               # = 4
q_table = np.zeros((states, actions))

for i in range(200):
    env.reset()
    state = 0
    done = False
```

얼음 호수 게임 버전 #2

```
while not done:
    if np.argmax(q_table[state]) > 0:      # 최대 Q값이 0보다 크면
        action = np.argmax(q_table[state]) # Q 테이블에서 가장 큰 값
    else:
        action = env.action_space.sample() # 랜덤 액션

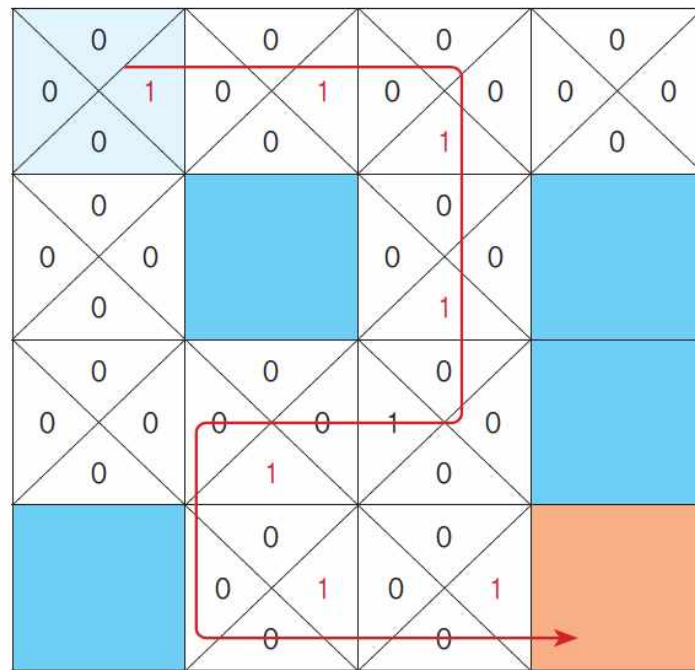
    new_state, reward, done, truncated, info = env.step(action)
    # 새로 얻은 정보로 Q-테이블 갱신
    q_table[state, action] = reward + np.max(q_table[new_state])
    state = new_state
    print(f'{i}번째 에피소드 후 q table=')
    print(q_table)
    env.render()
```

실행 결과

```
...  
199번째 에피소드 후 q table=  
[[0. 0. 1. 0.]  
 [0. 0. 1. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 0. 0.]  
 [0. 0. 0. 0.]  
 [0. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 1. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 0. 0.]  
 [0. 0. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 0.]
```

Q-학습 #2

- 문제점: 찾더라도 더 좋은 경로를 찾지 못한다.



탐험과 활용

- 어떻게 하면 새로운 경로를 찾을 수 있을까? 처음에는 **Q값**이 작은 액션이라고 하더라도 시도해 볼 필요가 있다. 이것을 강화학습에서는 탐험 (**exploration**)이라고 한다.
- 강화 학습에서도 처음에는 모험을 할 필요가 있다. 이것은 **e-greedy** 알고리즘으로 가능하다.

안가보던 길을 갈 수 있게 탐험을 한다.

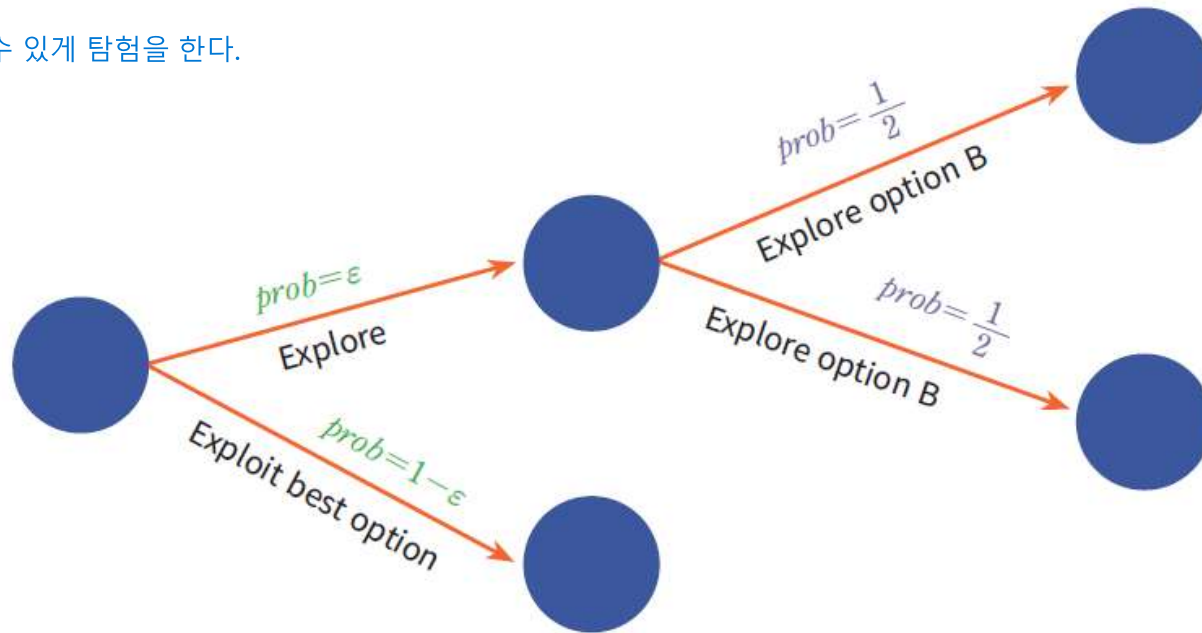


그림 17-16 e-greedy 알고리즘

e-greedy 알고리즘

- e-greedy 알고리즘에서는 **epsilon** 의 확률로 새로운 액션을 선택한다. (1-epsilon) 확률로 기존의 Q 값을 선택한다.
- 여기서 **epsilon** 은 처음에는 크게, 반복이 진행되면 점점 작게 하는 것이 관행이다.

```
for i in range(10000):  
    epsilon = 0.1/(i+1)  
    if random.random() < epsilon:  
        action = random           탐험 확률 안에 들어오면 액션을 랜덤하게 해준다.  
    else:  
        action = argmax(Q(s, a))  아니면 기존의 최고의 선택을 해주고
```



할인된 보상

- 에이전트가 미래에 받을 보상은 약간 할인해서 계산해야 한다.



총 보상

- 강화 학습에서도 “할인된 보상”이라는 개념을 사용한다. 미래의 보상에는 할인 계수 감마를 곱하여 총 보상을 계산한다. 할인 계수 γ 는 0에서 1 사이의 값이다.

$$R_t = r_t + \gamma r_{t+1} + \underbrace{\gamma^2 r_{t+2}}_{\text{할인}} + \dots$$

오래 걸리는 보상은 할인을 시켜야 한다.

할인(discount)된 보상

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$$

다음 상태에서 최대 보상 값의 할인율을 곱해서 준다.

할인된 보상이 필요한 이유

- 할인된 보상이 필요한 이유는 에이전트가 찾은 경로가 여러 개 있는 경우, 어떤 경로가 더 최단 경로인지를 판단해야 하기 때문이다
- 돌아가는 길은 더 할인되는 방향

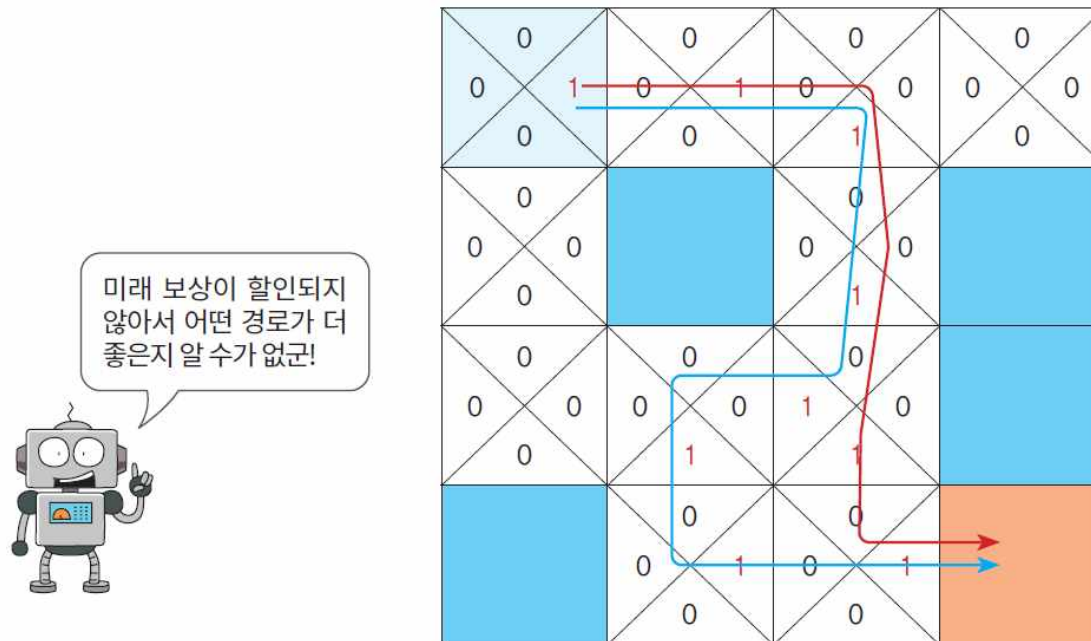
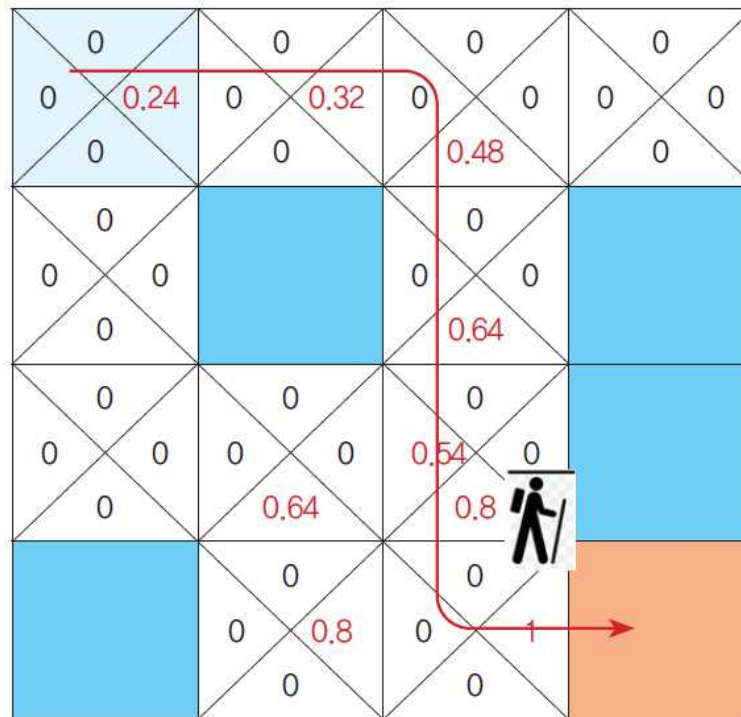


그림 17-18 할인되지 않은 보상을 사용하면 경로를 선택하기 어렵다.

할인이 있다면



더 짧은 길을 찾아낸다.

에이전트가 상태 s_{11} 에 있다면 DOWN 액션을 취할 수 있다.

그림 17-19 할인된 보상을 사용하면 최적의 경로를 찾을 수 있다.

확률적인 환경

- 앞에서 살펴본 Q-학습은 환경이 결정된 환경에서는 잘 작동한다. 하지만 확률적인 환경에서는 전혀 학습이 되지 않는다.
- 확률적인 환경이란 액션을 실행하였을 때 에이전트가 의도한 대로 가지 않을 수도 있는 환경이다.

실수로 아래키를 눌렀다
근데 아래 꺼를 학습하면 안되겠죠

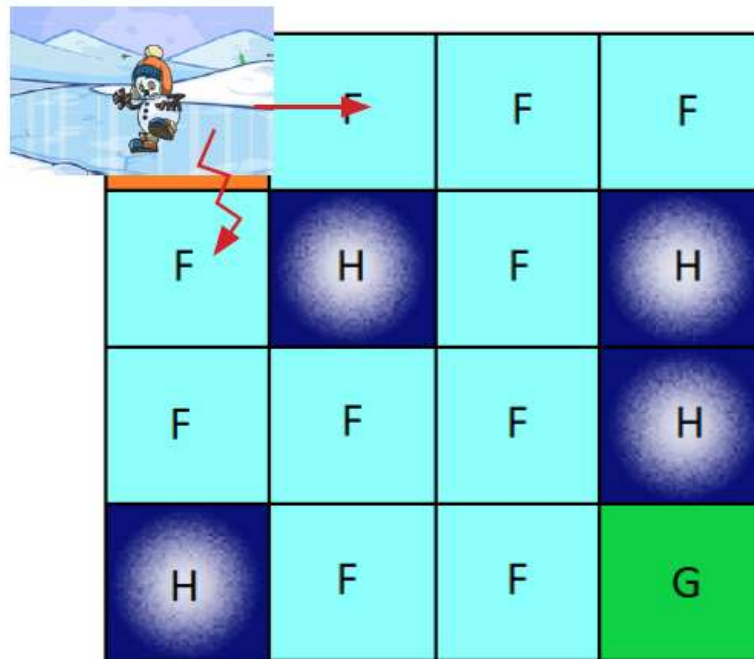


그림 17-20 강화 학습에서 기존의 값도 중요하다.

최종적인 Q 값 업데이트 방정식

$$Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a')]$$

의도하지 않은 경우 기존의 값 그대로 사용

기존의 Q값

새로운 Q값



$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

q 학습의 완성된 예

학습률

할인을 할인률을 곱해서 반영했다.

얼음 호수 게임 버전 #3

```
import gymnasium as gym
import numpy as np

env = gym.make("FrozenLake-v1", is_slippery=False)    # (1)

discount_factor = 0.9                                # (2)
epsilon = 0.9                                         # 탐험 확률
epsilon_decay_factor = 0.999                         # 탐험 확률 줄이기
learning_rate = 0.8
num_episodes = 1000                                  # 1000번의 에피소드로 훈련

states = env.observation_space.n                      # 16
actions = env.action_space.n                          # 4
q_table = np.zeros((states, actions))                 # (3)
```

얼음 호수 게임에서 Q-학습의 구현

```
for i in range(num_episodes):
    env.reset()
    state = 0
    epsilon *= epsilon_decay_factor      # 입실론: 탐사와 활용 비율 결정
    done = False

    while not done:                       # (4)
        if np.random.random() < epsilon: # 난수가 입실론보다 작으면 탐사
            action = env.action_space.sample() # 랜덤 액션
        else:                             # 난수가 입실론보다 작으면 활용
            action = np.argmax(q_table[state]) # Q 테이블에서 가장 큰 값

        new_state, reward, done, truncated, info = env.step(action)
        # 새로 얻은 정보로 Q-테이블 갱신
        q_table[state, action] += learning_rate * (reward + discount_factor *
np.max(q_table[new_state]) - q_table[state, action])
        state = new_state
    print(f'{i}번째 에피소드 후 q table=')
    print(q_table)
```

Q 테이블의 실제 모습

상태 \ 액션	↻	→	↰	⇅
상태 s0	0	0	0	0
상태 s1	0	0	0	0
상태 s2	0	0	0	0
상태 s3	0	0	0	0
...	...			

실행결과

```
...
999번째 에피소드 후 q table=
[[0.531441  0.59049  0.59049  0.531441 ]
 [0.531441  0.      0.6561   0.59049 ]
 [0.59049  0.729   0.59049  0.6561   ]
 [0.6561   0.      0.59048982 0.59049 ]
 [0.59049  0.6561   0.      0.531441 ]
 [0.      0.      0.      0.      ]
 [0.      0.81    0.      0.6561   ]
 [0.      0.      0.      0.      ]
 [0.6561   0.      0.729   0.59049 ]
 [0.6561   0.81    0.81    0.      ]
 [0.729    0.9     0.      0.729   ]
 [0.      0.      0.      0.      ]
 [0.      0.      0.      0.      ]
 [0.      0.80994816 0.9     0.729   ]
 [0.81     0.9     1.      0.81    ]
 [0.      0.      0.      0.      ]]
```

05 Deep Q-학습

Value Learning	Policy Learning
$Q(s, a)$ 를 계산한다.	$\pi(s)$ 를 찾는다.
$a = \max_{a'} Q(s, a')$	$\pi(s)$ 에서 액션 a 를 샘플링한다.

첫 번째 방법은 신경망이 Q 함수를 학습한다. 우리는 Q 함수로부터 액션을 결정할 수 있다.

두 번째 경우는 신경망이 직접적으로 정책을 학습한다. 정책에서 바로 액션을 결정한다. 두 번째 방법에서는 중간 단계의 Q 함수가 없다.

현대 강화학습에서는 policy-based(또는 actor-critic) 방법이 선호됩니다

왜 신경망을 사용하는가?

- 전통적인 Q-학습은 에이전트를 위한 치트 시트를 만드는 간단하지만 강력한 알고리즘이다. q 테이블을 그려야 한다
- 하지만 이 치트 시트가 너무 길면 어떻게 될까?
- 10,000개의 상태와 상태당 1,000개의 액션이 있는 환경을 상상해보자. 천만개의 셀을 가지는 Q-테이블이 필요하다. 해당 테이블을 저장하고 업데이트하는 데 필요한 메모리 양은 상태 수가 증가함에 따라 감당할 수 없을 만큼 증가한다.

천만개의 테이블을 행동할 때마다 업데이트 하면 버겁!
그래서 실제 게임에서는 못쓴다.



상태의 수

- 예를 들어서 100×100 크기의 화면을 가지고 있는 비디오 게임의 경우, 한 픽셀이 8바이트라고 하면 상태의 수는 얼마나 될까?
- 하나의 픽셀이 가질 수 있는 상태의 값은 256개이고 이러한 픽셀이 100×100 개나 있으므로 무려 $256^{100 \times 100}$ 이나 된다.
- 이렇게 탐색 공간이 무척 큰 경우가 바로 심층 신경망이 가장 필요한 경우이다.

그래서 q테이블 대신에 신경망으로 대신

DQN(Deep Q Network)

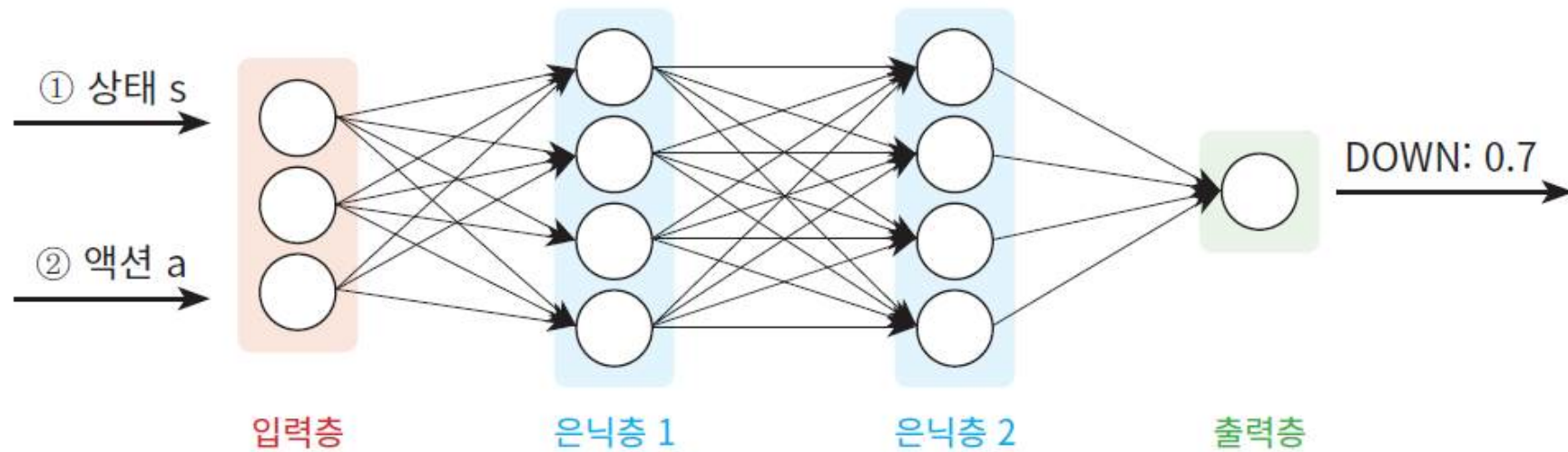


그림 17-21 DQN의 첫 번째 구조

출력으로 제일 높은 액션을 알려주거나
각 액션별로 확률을 알려준다

DQN(Deep Q Network)

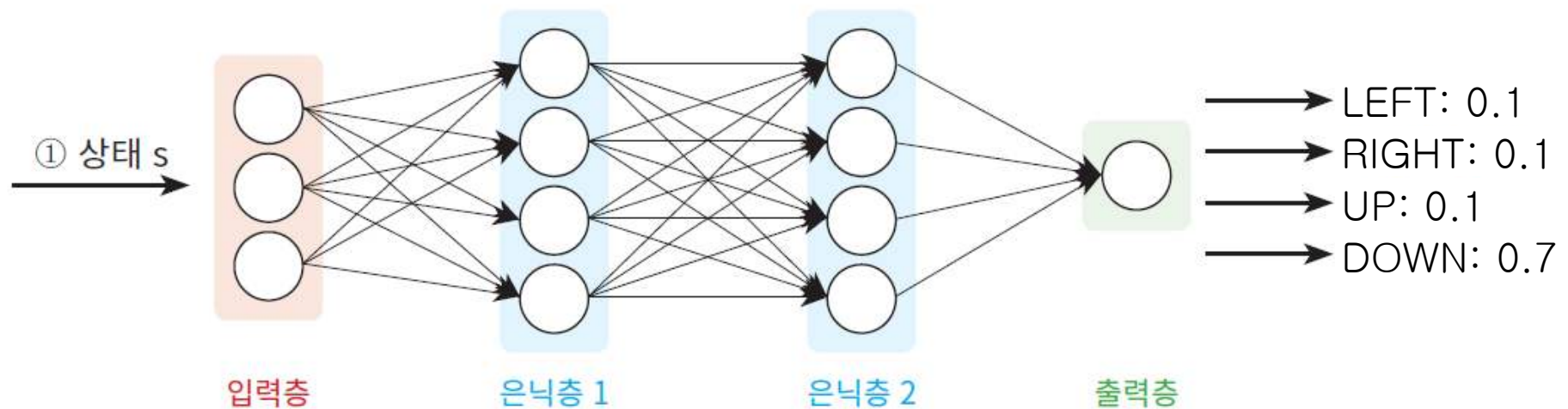


그림 17-22 DQN의 두 번째 구조

Q-학습 vs Deep Q-학습

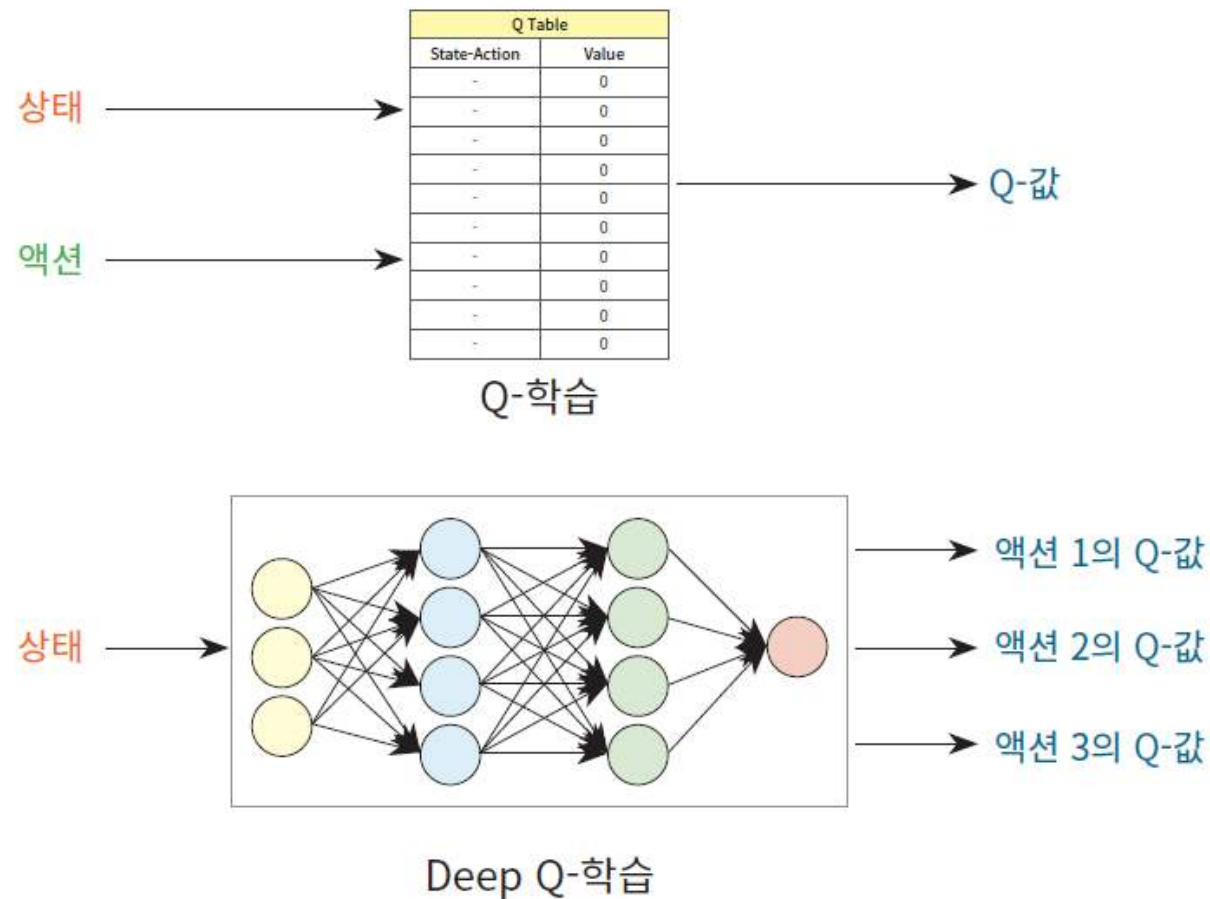


그림 17-23 Q-학습과 Deep Q-학습의 비교

학습 방법

오차 역전파 방법을 사용해서 학습을 한다, 그래서 오차가 필요하다.

- 선형 회귀 신경망을 통하여 생성된 출력값을 $Q(s, a)$ 라고 하자.
- 정답은 무엇일까? 특정한 액션 a 를 실행한 후라면 Q 값은 정의에 의하여 다음과 같이 변경되어야 한다. 이것이 정답이 된다.

현재 출력값이 정답이라고 가정하고,
기존의 q 값을 예측값으로 해서 이차이를 오차함수로 쓴

$$(r + \gamma \max_{a'} \hat{Q}(s', a'))$$

- 위의 값을 신경망이 생성한 Q 값과 비교하면서 차이를 줄이는 방향으로 가중치를 변경하면 된다.

$$E(\theta) = \sum_{t=0}^T \left[\underbrace{\hat{Q}(s_t, a_t | \theta)}_{\text{예측값(predicted)}} - \underbrace{\left(r_t + \gamma \max_{a'} \hat{Q}(s_{t+1}, a' | \theta) \right)}_{\text{목표값(target)}} \right]^2$$

예측값(predicted)

기존 테이블 값

목표값(target)

실행하여 개선된 목표값

이건 지도 함수의 접근법이다, 근데 지도 학습은 정답을 아는데 이건 아니다, 그래서 훈련이 잘 안된다.

알고리즘

$Q(s, a)$ 값을 난수로 초기화한다.

초기 상태 s 를 얻는다

for $t=1, T$ do

if 난수 $< \epsilon$ 액션 a_t 를 랜덤하게 선택한다.

else $a_t = \operatorname{argmax}_a Q^*(s_t, a | \theta)$

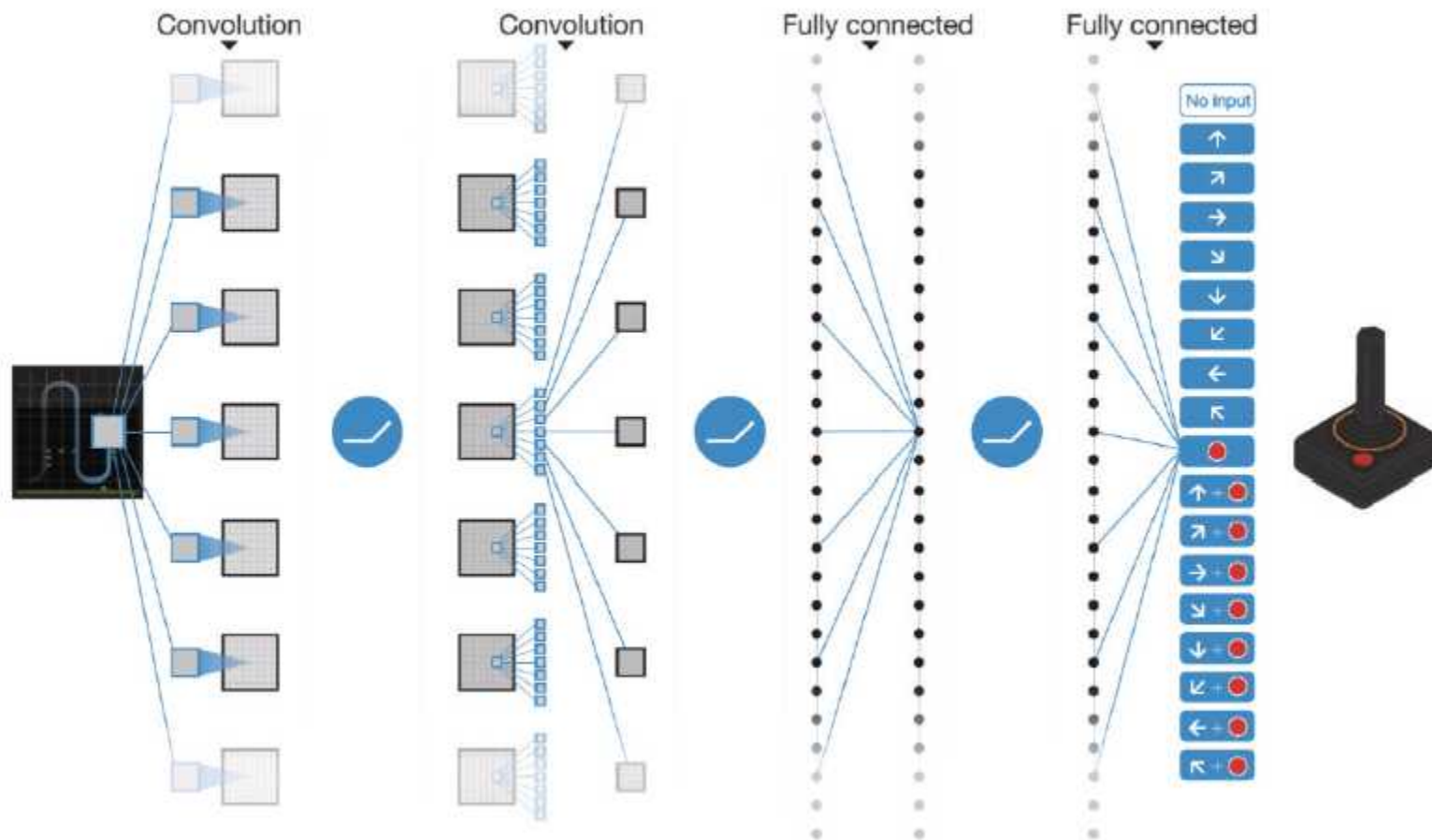
액션 a_t 를 실행하고 상태가 변경되고 보상 r_t 를 받는다.

$y_t = r_t + \gamma \operatorname{argmax}_a Q(s_{t+1}, a' | \theta)$

$(y_t - Q(s_t, a_t | \theta))^2$ 을 줄이기 위하여 경사하강법을 사용한다.

하나의 액션이 수행되고 보상과 다음 상태가 나왔기 때문에 보다 정확한 Q 값을 얻을 수 있다. 이것이 타겟(정확한 Q값)이 된다. 타겟과 현재 Q값의 차이를 이용하여 학습시킨다.

실제 적용 예: 벽돌 깨기 게임



신경망의 학습

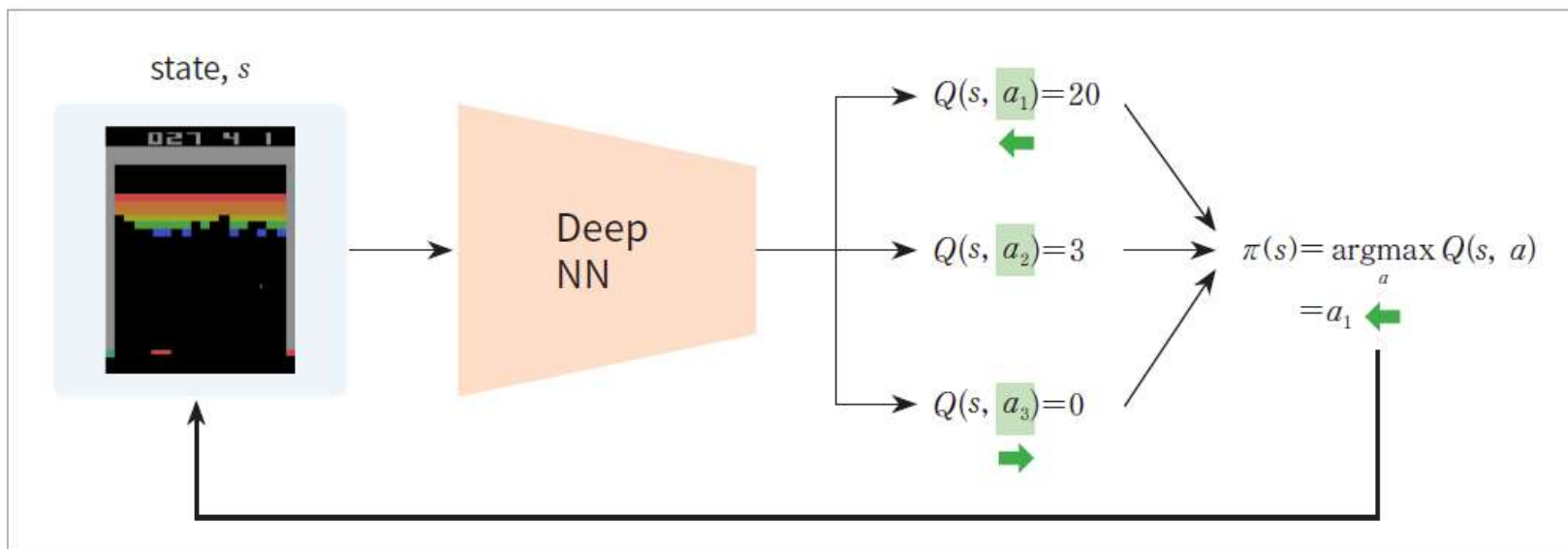
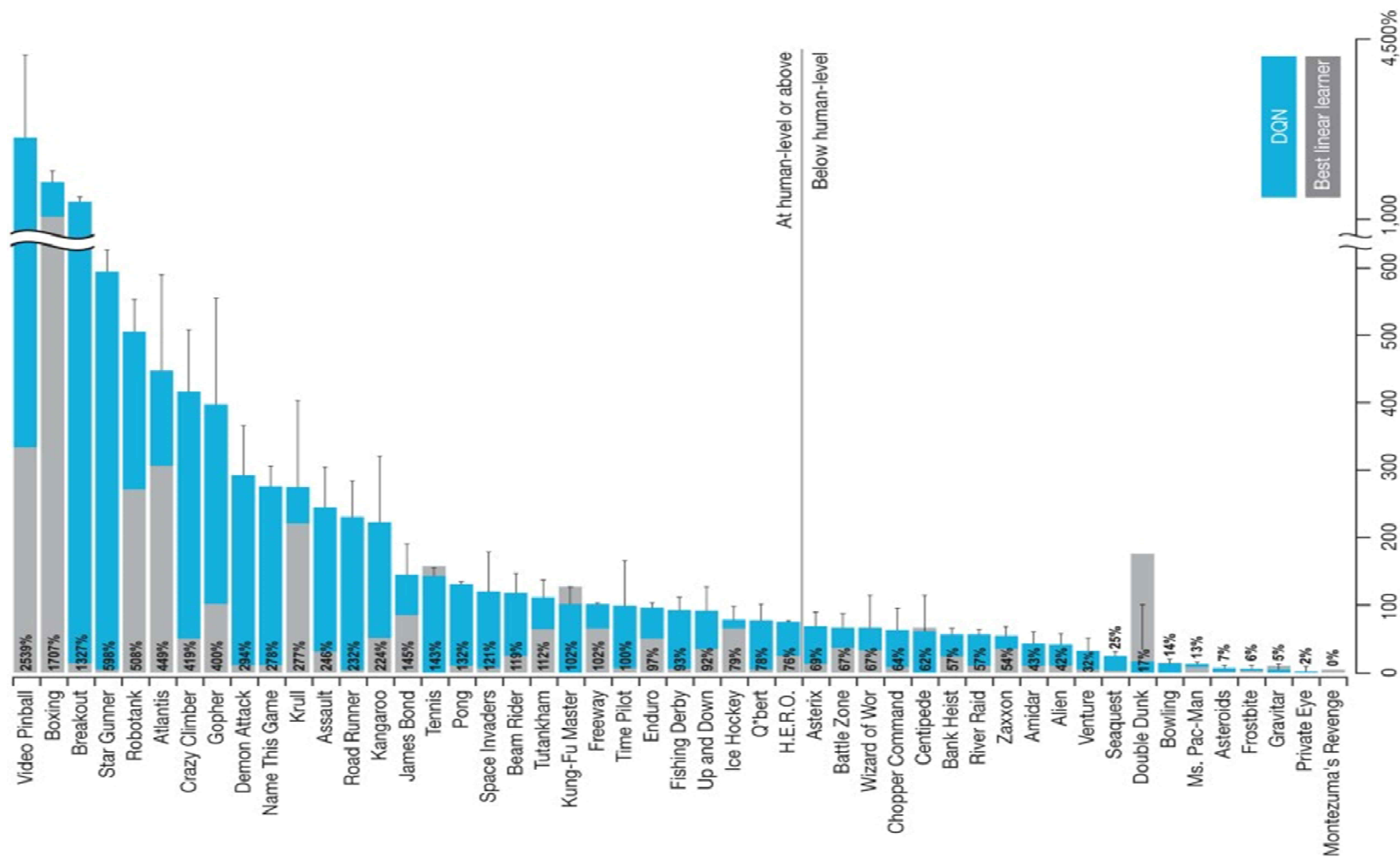


그림 17-25 Deep-Q 신경망에서의 학습

강화 학습을 이용한 게임의 성능



문제점

- Deep Q-학습에서는 약간의 문제가 있다. 우리는 목표 Q 값을 사실 정확히 알지 못한다. 그저 현재의 Q 값을 이용하여 추정할 뿐이다.
- 따라서 위의 알고리즘에서 볼 수 있듯이 반복할 때마다 목표가 변경된다.
- 이 문제를 해결하기 위하여 2개의 신경망을 사용하기도 한다.

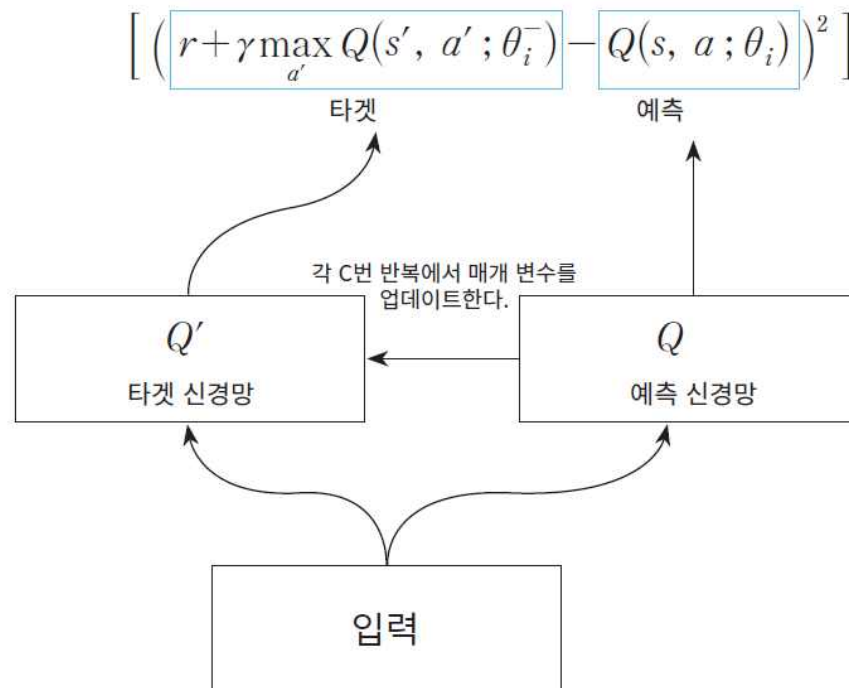


그림 17-27 타겟 신경망과 예측 신경망의 분리

심층 Q-학습의 단점

- 액션 공간이 비연속적이고 작을 때는 가능하지만 연속적인 액션 공간은 처리가 불가능하다. 행동이 몇 개 일 때만 가능하다
- 정책은 Q 함수로부터 결정적(deterministic)으로 계산된다. 따라서 확률적(stochastic)인 정책을 학습할 수 없다.
- Q-learning은 간단하고 이론적으로 깔끔하지만, 불안정성, 연속공간 한계, 탐색 효율 문제 때문에 현대 강화학습에서는 **policy-based**(또는 **actor-critic**) 방법이 선호됩니다.

비교 항목	Q-learning (DQN)	Policy Learning (PPO, SAC 등)
안정성	낮음 (발산 가능)	높음 (gradient-based update)
행동공간	이산형만 가능	연속형까지 가능
탐색	비효율적 (ϵ -greedy)	엔트로피 기반, 확률적 탐색
대표 사례	Atari 게임	로봇 제어, 자율주행, 전략 AI
최근 경향	거의 사용되지 않음	주류 방법

Policy Learning

- 직접, 정책 $\pi_{\theta}(a|s)$ 를 신경망으로 학습 (Q함수 없이 바로 행동 확률 출력)
 - 정책 그 자체를 파라미터화된 신경망 $\pi_{\theta}(a|s)$ 으로 나타내고, 보상을 최대화하는 방향으로 정책 경사(Policy Gradient)를 따라 직접 학습하는 강화 학습 패러다임이며, REINFORCE에서 시작해 PPO, SAC 등 현대 Actor-Critic 알고리즘까지 포괄

세대	알고리즘	특징	점유율
1세대	REINFORCE	정책 경사(Policy Gradient) 기본	거의 안 씀
2세대	Actor-Critic (A2C/A3C)	Actor(정책) + Critic(가치함수)로 분산 줄임	중간
3세대	TROP / PPO	신뢰 영역(Trust Region)으로 안정성 확보	70% 이상
	SAC, TD3	연속 행동 공간 + 최대 엔트로피 -> 로봇 제어에 유리	20%, 게임
4세대	Offline RL (CQL, IQL 등)	실제 데이터만으로 학습 (시뮬레이션 없이)	급성장 중

- 실무에서 많이 쓰이는 알고리즘

순위	알고리즘	행동 공간	대표 사용처
1	PPO	이산/연속 모두	OpenAI Gym, Unity ML-Agents, DeepMind, Tesla FSD
2	SAC	연속 전용	Google Robotics, NVIDIA Isaac, Wayve, 자율 주행
3	DQN / Rainbow	이산 전용	게임 AI, 광고 추천, 카카오/네이버 추천
4	A2C / A3C	이산/연속	멀티 에이전트, 스타크래프트 AI
5	Offline RL (CQL, IQL)	오프라인 데이터	실제 병원 데이터, 금융 트레이딩

항목	Q-Learning 계열 (Value-based)	Policy Learning 계열 (Policy-based)
학습 대상	$Q(s, a)$ 또는 $V(s)$	정책 $\pi_{\theta}(a s)$
행동 선택 방식	ϵ -greedy 또는 $\operatorname{argmax} Q$	확률적으로 뽑음 (softmax)
연속 행동 가능?	불가능 (DQN도 이산만)	가능 (PPO, SAC)
샘플링 효율	높음 (OffPolicy 가능)	낮음(OnPolicy) -> 경험 재활용 어려움 PPO/SAC로 개선
학습 안정성	매우 안정적 (타겟 네트워크로 해결)	불안정 -> PPO(clip), SAC(entropy)로 해결
실무 점유율 (2025)	20% (게임, 이산 행동)	80% (로봇, 연속, 복잡 환경)

Policy-based 강화학습

- 핵심 아이디어 3단계

- 1. 파라미터화된 정책

$\pi_{\theta}(a|s)$ = 신경망이 상태 s 를 보고 행동 a 를 선택할 확률을 출력

→ θ = 신경망의 모든 weight & bias

- 2. 정책 경사(Policy Gradient) 정리를 이용한 업데이트

$$\nabla_{\theta} J(\theta) = E[\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot Q^{\pi_{\theta}}(s_t, a_t)]$$

→ “이 행동을 더 자주 했으면 보상이 더 올라갈까?”를 수학적으로 계산

- 3. 목표: 보상 기대값 $J(\theta)$ 를 최대화

$$J(\theta) = E_{\tau \sim \pi_{\theta}}[\sum \gamma^t r_t]$$

→ $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$ (기본 방향)

PPO (Proximal Policy Optimization)

- 핵심 문제 해결: 기존 Policy Gradient는 한 번에 너무 크게 θ 를 바꾸면 성능이 폭락 $\rightarrow$ 학습 불안정
- **PPO**의 해결책: “너무 크게 바뀌지 않게 강제로 제한하자!”
 - “너무 크게 정책을 바꾸면 성능이 깨지니까, 변화 비율을 $\epsilon=0.2$ 안에서만 믿고 나머지는 무시하자!”

요소	수식·설명	왜 중요한가?
Clipped Surrogate Objective	$L(\theta) = E[\min(r(\theta) \hat{A}, \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon) \hat{A})], r(\theta) = \pi_{\theta}(a s) / \pi_{\theta_{\text{old}}}(a s)$	
Advantage $\hat{A}$	$A(s,a) = Q(s,a) - V(s)$ 또는 GAE(λ) 사용	“이 행동이 평균보다 얼마나 좋았는가?”를 정확히 측정
Value Loss 추가	$+ c1 (V_{\theta}(s) - V_{\text{target}})^2 + c2 S \pi_{\theta}$	Critic도 같이 학습 $\rightarrow$ 분산 감소 + 엔트로피 보너스
결과	학습이 거의 안 깨짐 + 성능 매우 높음	2025년 기준 실무 55~60% 점유율

SAC (Soft Actor-Critic)

- 핵심 문제 해결:
 1. 결정론적 정책은 탐색을 못 함
 2. α (엔트로피 계수)를 수동으로 조절하면 귀찮음
- **SAC**의 천재적인 해결책:

“탐색(엔트로피)을 보상의 일부로 만들어 자동으로 최적 탐색-활용 균형을 맞추고, α 도 학습하자!”
- “**SAC**는 최대 엔트로피 강화학습으로, 보상에 엔트로피 보너스를 추가해 자연스러운 탐색을 유도하고, 엔트로피 계수 α 까지 자동으로 학습하게 함으로써 연속 행동 공간에서 최고의 샘플 효율과 성능을 달성하는 알고리즘입니다.”

- **SAC의 학습 과정 (5단계)**

1. 현재 정책 π_{θ} 로 행동 a 를 뽑음 $\rightarrow$ 정규분포로 부드럽게 나눔
 $a \sim N(\mu_{\theta}(s), \sigma_{\theta}(s))$
2. 환경에서 보상 r 받고 다음 상태 s' 로 감
3. Critic이 “이 행동이 얼마나 좋았을까?”를 2개 Q함수로 평가
 $\rightarrow \min(Q1, Q2)$ 써서 과대평가 막음
4. 엔트로피 보너스 계산
추가 보상 $= r + \alpha \times H(\pi)$
(H 가 클수록 = 행동이 다양할수록 보너스 $\uparrow$)
5. α 도 같이 학습
목표: 엔트로피가 목표치(-행동차원) 근처로 자동 유지
 $\rightarrow$ 처음엔 많이 탐험, 나중에 적당히 집중

요소	수식·설명	왜 중요한가?
최대 엔트로피 목표	$J(\theta) = E[\sum (r(s,a) + \alpha H(\pi(\cdot s)))]$	
자동 α 조절	α 를 학습 가능한 파라미터로 만들고, 목표 엔트로피 $H_{target} = -\dim(A)$ 를 맞추도록 학습	사람이 α 안 건드려도 됨 → 완전 자동
Q 함수를 2개 씀 (Clipped Double Q)	$Q(s,a) \leftarrow r + \gamma (1-d) [\min(Q1, Q2)(s',a') + \alpha H(\pi(\cdot s'))]$	
정책은 평균·분산 분리	$\pi_{\theta}(a s) = N(\mu_{\theta}(s), \sigma_{\theta}(s))$	→ reparameterization trick으로 기울기 계산 가능
결과	샘플 효율 극강 + 연속 행동에서 최고 성능 + α 자동 조절	로봇 제어·자율주행 실무 1위

Summary

- 강화 학습(**Reinforcement Learning**)에서는 에이전트가 어떤 행동을 취할 때마다 외부에서 처벌이나 보상이 주어진다. 컴퓨터는 이 보상을 최대화하는 방향으로 학습을 진행한다.
- 상태(**state**) 는 에이전트의 현재 상황이다. 보상(**reward**) 은 환경으로부터의 피드백이다. 액션(**action**) 는 에이전트의 행동이다.
- 강화 학습에서 보상은 “할인된 보상”이다. 미래의 보상에는 할인 계수 감마를 곱하여 총 보상을 계산한다.
- **Q** 함수는 상태 **s**에 있는 에이전트가 어떤 액션 **a**를 실행하여서 얻을 수 있는 미래 보상값의 기대값이다.
- 탐사는 가끔 모험을 하는 것이고 활용은 기존의 **Q** 값을 사용하는 것이다.
- 심층 **Q**-학습은 **Q** 값을 심층 신경망을 이용하여 학습한다. 우리는 신경망을 사용하여 이러한 **Q** 값을 근사할 수 있다. 이러한 신경망을 **DQN(Deep Q Network)**이라고 한다.

Q & A

